# The Complete Guide to Measuring What Matters in Generative RAG AI Systems

9 min read · Oct 14, 2025

Dewasheesh Rana



## 🧭 Introduction — From Smart to Measurable AI

Retrieval-Augmented Generation (**RAG**) is at the heart of today's GenAI systems — from copilots that debug code to assistants that explain financial transactions or resolve incidents.

**But here's the truth:**

> A RAG pipeline that *sounds smart* isn't necessarily *doing its job right.*

To build AI systems that are reliable, explainable, and production-grade, you must **measure** their behavior — not just their accuracy.

In this article, we'll dive into the full landscape of **RAG and Agentic RAG metrics** — starting from the basics like Precision and Recall, then moving to advanced measures like nDCG, Faithfulness, Context Precision, and RAGAS — with detailed numerical examples anyone can understand.

By the end, you'll know exactly how to evaluate a GenAI system the same way you'd test a financial algorithm or cloud microservice: with hard, measurable evidence.

## 🧩 Understanding the RAG Pipeline

Before we discuss metrics, let's recall the architecture:

```
User Query → Retriever → Context → Generator → Final Answer
```

The **Retriever** fetches relevant data (from vector DBs like Qdrant, OpenSearch, Pinecone, etc.).

The **Generator** (LLM) reads this context and produces the answer.

Together, they form the **RAG pipeline**.

Agentic RAG extends this with **autonomous decision-making** — where agents (like CrewAI) can:

- invoke tools,



- execute corrective actions.

So, metrics matter even more — because now, your AI is *acting* as well as *thinking*.

## 🔍 PART 1 — Retrieval Metrics

### How good is your "librarian"?

Retrieval metrics evaluate the quality of the context your LLM is fed.
If you feed garbage, you'll get confident-sounding garbage back.

### 🧠 Example

Query: *"Why did the Airflow DAG fail yesterday?"*

Retriever fetches 5 log snippets:

| Rank | Log Snippet | Relevant? |
|---|---|---|
| 1 | Timeout error on operator | ✅ |
| 2 | Scheduler heartbeat | ❌ |
| 3 | DB connection reset | ✅ |
| 4 | Memory warning | ✅ |
| 5 | Debug trace | ❌ |

There are 4 true relevant logs in total.

## 🎯 Precision@K — "Purity"

How many of the retrieved docs are relevant?

$$Precision@5 = \frac{relevant\ retrieved}{total\ retrieved} = 3/5 = 0.6$$

✅ 60% of what we retrieved is useful.

⚠️ Low Precision = noisy retrieval → wasted tokens.

## 🎯 Recall@K — "Coverage"

How many of the relevant docs did we actually find?

$$Recall@5 = \frac{relevant\ retrieved}{total\ relevant} = 3/4 = 0.75$$

✅ We covered 75% of the right info.

⚠️ Low Recall = missing facts → model will hallucinate.

## 🎯 MRR (Mean Reciprocal Rank)

How early do the correct results appear?

If the first relevant doc appears at rank 2 → Reciprocal Rank = 1/2 = 0.5

Average across queries to get **MRR.**
✅ Higher MRR → users find answers faster.

## 🎯 nDCG (Normalized Discounted Cumulative Gain)

**The most powerful retrieval metric.**

It rewards ranking quality — relevant documents higher in the list matter more.

$$DCG@K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

$$nDCG@K = \frac{DCG@K}{IDCG@K}$$

Example:

| Rank | rel | Contribution |
|---|---|---|
| 1 | 3 | 7/1 = 7.00 |
| 2 | 2 | 3/1.58 = 1.89 |
| 3 | 1 | 1/2 = 0.5 |

DCG@5 = 9.39
If perfect → IDCG@5 = 9.39
nDCG = 1.0 ✅ perfect ranking.

If order shuffled → DCG = 6.9 → nDCG = 0.74

✅ nDCG shows *how close your ranking is to ideal.*

## 🧭 Understanding nDCG — Normalized Discounted Cumulative Gain

**nDCG** measures how good your ranking is compared to the *best possible (ideal) ranking*.
It's the gold-standard metric for **search engines, recommendation systems, and RAG retrievers.**

### ⚙️ Step 1: Formula

**Discounted Cumulative Gain (DCG):**

```
DCG@K = Σ ( (2^rel_i - 1) / log2(i + 1) )
```

where

- `rel_i` = relevance score of item *i*
- `i` = rank position (1-based)
- `K` = number of results you evaluate

**Normalized DCG (nDCG):**

```
nDCG@K = DCG@K / IDCG@K
```

`IDCG@K` is the ideal DCG if results were perfectly ranked.

### ⚙️ Step 2: Example — Perfect Ranking

#### rel (Relevance)

How relevant the item is (usually human-judged)

- `3` = highly relevant

- `2` = relevant
- `1` = somewhat relevant
- `0` = not relevant

```
Rank | rel | Contribution Formula                    | Contribution
-----|-----|-----------------------------------------|--------------
1    | 3   | (2^3-1)/log2(1+1) = 7/1               | 7.00
2    | 2   | (2^2-1)/log2(2+1) = 3/1.58            | 1.89
3    | 1   | (2^1-1)/log2(3+1) = 1/2               | 0.50
```

```
DCG@3 = 7 + 1.89 + 0.50 = 9.39
IDCG@3 = 9.39
nDCG@3 = 9.39 / 9.39 = 1.0 ✅ perfect ranking
```

### ⚙️ Step 3: Example — Shuffled Ranking

Suppose the ranking is wrong:

```
Rank | rel
-----|-----
1    | 1
2    | 3
3    | 2
```

Compute DCG:

```
(2^1-1)/log2(1+1) = 1/1    = 1.00
(2^3-1)/log2(2+1) = 7/1.58 = 4.43
(2^2-1)/log2(3+1) = 3/2    = 1.50
```

```
DCG@3 = 1 + 4.43 + 1.50 = 6.93
IDCG@3 = 9.39
nDCG@3 = 6.93 / 9.39 = 0.74
```

✅ **nDCG = 0.74 → 74% as good as the ideal ranking**

## ✅ Key Takeaways

- **nDCG = 1.0 → perfect ranking**
- **nDCG ≈ 0 → poor ranking**
- High-relevance items at top positions boost the score exponentially
- Log discount mimics human behavior — users care more about top results

## 🧩 Retrieval Metrics Summary

🧩 Retrieval Metrics Summary

| Metric | Focus | Ideal |
|---|---|---|
| Precision@K | % of top K results relevant | ≥ 0.7 |
| Recall@K | % of all relevant results retrieved | ≥ 0.8 |
| MRR | Early relevance | ≥ 0.5 |
| nDCG | Overall ranking quality | ≥ 0.85 |

## ✍️ PART 2 — Generation Metrics

### How good is your “writer”?

The retriever feeds the context; now the LLM must produce an answer that’s **grounded, relevant, and complete.**

### 🎯 Generation Precision — "Groundedness"

How much of what the model said is actually supported by the context?

$$GenerationPrecision = \frac{supported\ statements}{total\ statements}$$

Example:

LLM generated 10 sentences → 8 supported → **Precision = 0.8**

✅ Higher = fewer hallucinations.

⚠️ Low = model is making things up.

### 🎯 Generation Recall — "Completeness"

Of all relevant facts available, how many did the LLM actually use?

$$GenerationRecall = \frac{relevant\ context\ used}{total\ relevant\ context}$$

Example:

10 relevant facts in retrieved chunks → LLM used 7 → Recall = 0.7

✅ Higher = more comprehensive.

⚠️ Low = answer might miss key details.

### 🎯 Faithfulness

Checks that all statements are consistent with context (no contradictions).

If 9 of 10 statements can be verified → Faithfulness = 0.9 ✅

### 🎯 Context Precision — "Did we give the model too much junk?"

How efficiently the LLM used the retrieved context.

$$ContextPrecision = \frac{used\ chunks}{retrieved\ chunks}$$

Retriever fetched 5 chunks; LLM used 3 → **0.6**

⚠️ Low = too many irrelevant chunks (optimize retriever).

### 🎯 Answer Relevance — "Did it answer the question?"

How semantically aligned is the generated answer with the user's query?

$$AnswerRelevance = cosine\_sim(query, answer)$$

> *Use **cosine similarity** or **LLM scoring**:*

Example:

LLM output = "It failed due to timeout."

Ground Truth = "Timeout & DB reset."

→ Relevance = 0.9 ✅

If output = "Airflow is a scheduler." → 0.1 ❌

### 🧠 Generation Metric Summary

🧠 **Generation Metric Summary**

| Metric | Focus | What It Measures | Ideal |
|---|---|---|---|
| Generation Precision | Faithfulness | % statements grounded in context | ≥ 0.8 |
| Generation Recall | Completeness | % of context facts used | ≥ 0.7 |
| Context Precision | Efficiency | % of retrieved chunks used | ≥ 0.6 |
| Answer Relevance | Intent match | Query ↔ Answer alignment | ≥ 0.85 |

## 🤖 PART 3 — Agentic RAG Metrics

### When your AI starts reasoning and acting

Agentic RAG introduces **autonomous reasoning and tool execution**, like CrewAI or LangGraph agents calling APIs or validating results.

So, you add a **new metric layer: Action-level metrics.**

### 🎯 Tool Accuracy

How many agentic actions (like database queries or log fetches) succeeded?

$$ToolAccuracy = \frac{successful\ actions}{total\ actions}$$

If 9 out of 10 tool calls worked → **0.9**

### 🎯 Reasoning Trace Consistency

How consistent was the agent's step-by-step reasoning with retrieved evidence?

LLM evaluators (or humans) check:

- Did the agent cite facts correctly?
- Were actions logically sequenced?

Typical score range: 0 → 1

### 🎯 Multi-Hop Answer Faithfulness

In Agentic RAG, the model might:

*Retrieve data*

*Query another tool*

*Generate the final answer*

Each hop compounds risk.
You measure **Faithfulness per hop**, then aggregate.

Example:
Hop 1 (Retrieval): 0.9
Hop 2 (Computation): 0.8
Hop 3 (Final Answer): 0.85
→ Avg Faithfulness = 0.85 ✅

### 📊 PART 4 — RAGAS (Comprehensive Evaluation)

**RAGAS (Retrieval-Augmented Generation Assessment Suite)** combines all the above into one interpretable score.

$$RAGAS = 0.4 \times Faithfulness + 0.2 \times ContextPrecision + 0.2 \times ContextRecall + 0.2 \times AnswerRelevance$$

Example:
Faithfulness = 0.9
ContextPrecision = 0.7
ContextRecall = 0.8
AnswerRelevance = 0.9
→ RAGAS = 0.84 ✅ Production-ready

## 🧠 How These Metrics Differ Between RAG and Agentic RAG

| Layer | RAG | Agentic RAG |
|---|---|---|
| **Retrieval Metrics** | Precision, Recall, nDCG | Same |
| **Generation Metrics** | Faithfulness, Answer Relevance | Same + Reasoning validation |
| **Action Metrics** | N/A | Tool accuracy, multi-hop consistency |
| **System Metrics** | RAGAS | Extended RAGAS (includes agent hops) |
| **Optimization Goal** | Accuracy & grounding | Accuracy + autonomy + safety |

✅ RAG = Knows the right thing
🤖 Agentic RAG = Knows + Does the right thing

## 📈 PART 5 — Production Integration

You can integrate all these metrics in your MLOps stack using:

| Tool | Purpose |
|---|---|
| **RAGAS** | Grounding & relevance |
| **TruLens** | Real-time evaluation dashboard |
| **LangSmith** | Traces & dataset analytics |
| **Evidently AI + Prometheus** | Drift & stability monitoring |
| **Grafana** | System metrics visualization |

## ⚙️ Example: Evaluating an Incident Copilot

| Metric | Value | Target | Meaning |
|---|---|---|---|
| Recall@5 | 0.84 | ≥ 0.8 | Key logs retrieved |
| Precision@5 | 0.68 | ≥ 0.7 | Some noise |
| Faithfulness | 0.9 | ≥ 0.8 | Grounded output |
| Generation Recall | 0.75 | ≥ 0.7 | Sufficient completeness |
| Answer Relevance | 0.91 | ≥ 0.85 | Aligned with query |
| RAGAS | 0.84 | ≥ 0.8 | Overall strong |
| MTTR Reduction | 60% | — | Proven business impact |

## 🧭 Key Takeaways

- **Retrieval metrics** (Precision, Recall, nDCG) tell you *if the right facts were found.*
- **Generation metrics** (Faithfulness, Context Precision, Answer Relevance) tell you *if the LLM used and explained them correctly.*
- **Agentic RAG metrics** add *tool accuracy and reasoning consistency.*
- **RAGAS** ties it all together into a single, trackable score.

✅ High precision → trustworthy answers
✅ High recall → complete answers
✅ High relevance → helpful answers
✅ High faithfulness → safe answers

> **A great GenAI system doesn't just answer questions — it proves it knows *why* its answer is correct.**

## ✅ Official RAGAS Metrics

### 🎯 1. `context_precision`

**Definition (RAGAS):**
Measures the proportion of retrieved contexts that are relevant to the ground truth answer.

- Evaluates retriever quality.
- Higher = fewer irrelevant chunks retrieved.
- Does NOT measure how the LLM "used" the context.

✓ Retriever metric
✗ Not a generation metric

### 🎯 2. `context_recall`

**Definition (RAGAS):**
Measures whether all relevant information needed to answer the question was retrieved.

- Higher = retrieval covers necessary facts.
- Lower = retriever missed important context.

✓ Retriever metric

## 🎯 3. `faithfulness`

**Definition (RAGAS):**
Measures whether the generated answer is fully supported by the retrieved context.

- Detects hallucinations.
- Checks grounding of answer to context.

✓ Generation metric
✓ Groundedness metric

## 🎯 4. `answer_correctness`

**Definition (RAGAS):**
Measures semantic similarity between the generated answer and the ground truth answer.

- Evaluates factual correctness.
- Independent of whether hallucination occurred.

✓ Generation metric

## 🎯 5. `answer_relevancy` (sometimes called answer relevance)

**Definition (RAGAS):**
Measures whether the generated answer addresses the user's question.

- Detects off-topic responses.

✓ Generation metric

## ❌ What Is NOT Official in RAGAS

These are NOT official RAGAS metrics:

- "Generation Precision"
- "Generation Recall"
- "LLM used 3 out of 5 chunks"

Those are conceptual explanations, not implemented RAGAS metrics.

## ✅ Correct Categorization

### Retriever Metrics

- `context_precision`
- `context_recall`

### Generation Metrics

- `faithfulness`
- `answer_correctness`
- `answer_relevancy`

### Perfect — let's compare them cleanly and formally.

## 🎯 Precision@K vs RAGAS `context_precision`

| Aspect | Precision@K (IR) | RAGAS `context_precision` |
|---|---|---|
| Field | Information Retrieval | RAG Evaluation |
| Evaluates | Retriever | Retriever (semantic) |
| Requires manual relevance labels? | ✅ Yes | ❌ No |
| Uses ground-truth answer? | ❌ No | ✅ Yes |
| Uses LLM / embeddings? | ❌ No | ✅ Yes |
| Mathematical formula | Relevant@K / K | Semantically relevant contexts / K |
| Measures | Retrieval purity | Retrieval usefulness for answering |
| Deterministic? | Yes (given labels) | No (depends on semantic judge) |

## 🔎 1️⃣ Precision@K (Classic IR)

### Definition

$$Precision@K = \frac{\text{Number of relevant documents in top K}}{K}$$

You must already know which documents are relevant.

### Example

Top 5 retrieved docs:

- 3 are labeled relevant
- 2 are irrelevant

[Precision@5 = 3/5 = 0.6]

✓ Pure retriever metric
✓ No generation involved

## 🔎 2️⃣ RAGAS `context_precision`

### Definition (Conceptual)

Measures how many retrieved contexts are relevant to producing the **ground-truth answer.**

Instead of manual labels, RAGAS:

- Uses semantic similarity
- Uses embeddings and/or LLM judgment
- Evaluates relevance relative to the ground truth answer

### Example

Top 5 retrieved chunks:

- 3 contain information needed for ground truth
- 2 are unrelated

Score ≈ 0.6

But the relevance is determined semantically, not manually.

## 🧠 Key Conceptual Difference

Precision@K asks:

> "Is this document relevant to the question?"

RAGAS `context_precision` asks:

> "Was this retrieved context useful for generating the correct answer?"

Subtle but important difference.

### 📌 When They Match

If:

- You have perfect manual labels
- RAGAS semantic judge is accurate

Then:

```
Precision@K ≈ context_precision
```

But they are computed differently.

### 📌 When They Differ

They may differ when:

- A document is related to the question but not needed for the answer.
- A document partially overlaps semantically.
- Semantic similarity model misjudges relevance.

### 🎯 Clean Mental Model

Precision@K = Hard-labeled document relevance
RAGAS context_precision = Soft semantic answer-support relevance

## Written by Dewasheesh Rana

505 followers · 2 following

Dewasheesh Rana is an AI/ML Architect with 14+ yrs experience in Agentic AI, LLMs, RAG, MLOps, API's building scalable systems on Azure, AWS and PCF at scale.

## No responses yet

Ramesh Chukka

What are your thoughts?